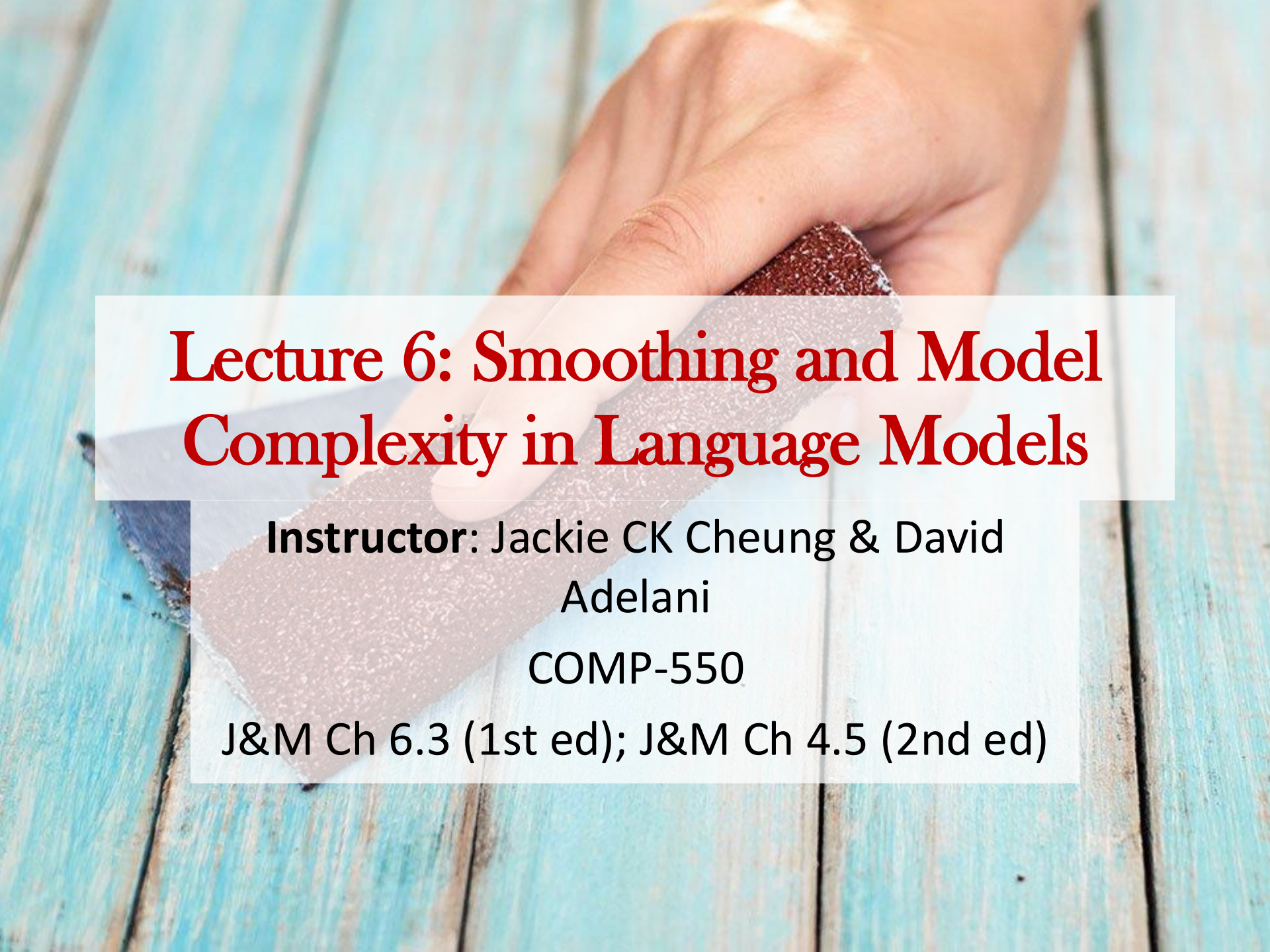


A close-up photograph of a hand holding a rectangular block of dark brown sandpaper, sanding a piece of light-colored wood. The background consists of horizontal wooden planks painted in a vibrant blue color. The sandpaper is being moved across the wood, creating a fine, light-colored dust. The hand is positioned in the upper right, with the thumb and index finger visible, gripping the sandpaper block.

Lecture 6: Smoothing and Model Complexity in Language Models

Instructor: Jackie CK Cheung & David Adelani

COMP-550

J&M Ch 6.3 (1st ed); J&M Ch 4.5 (2nd ed)

In This Lecture We Will...

- Evaluate language models with perplexity
- Justify the choice of using relative frequencies as **maximum likelihood estimation (MLE)**
- Discuss problems with MLE
- Have language models deal with unseen data via **smoothing**

Language Modelling

Predict the next word given some context

Mary had a little _____

- *lamb* GOOD
- *accident* GOOD?
- *very* BAD
- *up* BAD

N-grams

Make a **conditional independence assumption** to make the job of learning the probability distribution easier.

- Context = the previous N-1 words

Common choices: N is between 1 and 3

Unigram model

$$P(w_N|C) = P(w_N)$$

Bigram model

$$P(w_N|C) = P(w_N|w_{N-1})$$

Trigram model

$$P(w_N|C) = P(w_N|w_{N-1}, w_{N-2})$$

Deriving Parameters from Counts

Simplest method: count N-gram frequencies, then divide by the total count

e.g.,

Unigram: $P(\text{cats}) = \text{Count}(\text{cats}) / \text{Count}(\text{all words in corpus})$

Bigram: $P(\text{cats} \mid \text{the}) = \text{Count}(\text{the cats}) / \text{Count}(\text{the})$

Trigram: $P(\text{cats} \mid \text{feed the}) = ?$

These are the **maximum likelihood estimates (MLE)**.

Evaluation Measures

Likelihood of generating the test corpus

i.e., $P(\text{test_corpus}; \theta)$, where θ represents the parameters learned by training our LM on the training data

Intuition: a good language model should give a high probability of generating some new, valid English text.

Absolute number is not very meaningful—this can only be used to compare the quality of different language models!

Unwieldy because of small values, so not actually used in the literature. Alternatives to likelihood:

Cross-entropy

Perplexity

Basic Information Theory

Consider some random variable X , distributed according to some probability distribution.

We can define information in terms of how much certainty we gain from knowing the value of X .

Rank the following in terms of how much information we expect to gain by knowing its value:

- Fair coin flip

- An unfair coin flip where we get tails $\frac{3}{4}$ of the time

- A very unfair coin that always comes up heads

Likely vs Unlikely Outcomes

Observing a likely outcome – less information gained

Intuition: you kinda knew it would happen anyway

- e.g., observing the word *the*

Observing a rare outcome: more information gained!

Intuition: it's a bit surprising to see something unusual!

- e.g., observing the word *armadillo*

Formal definition of information in bits:

$$I(x) = \log_2\left(\frac{1}{P(x)}\right)$$

Minimum number of bits needed to communicate some outcome x

Entropy

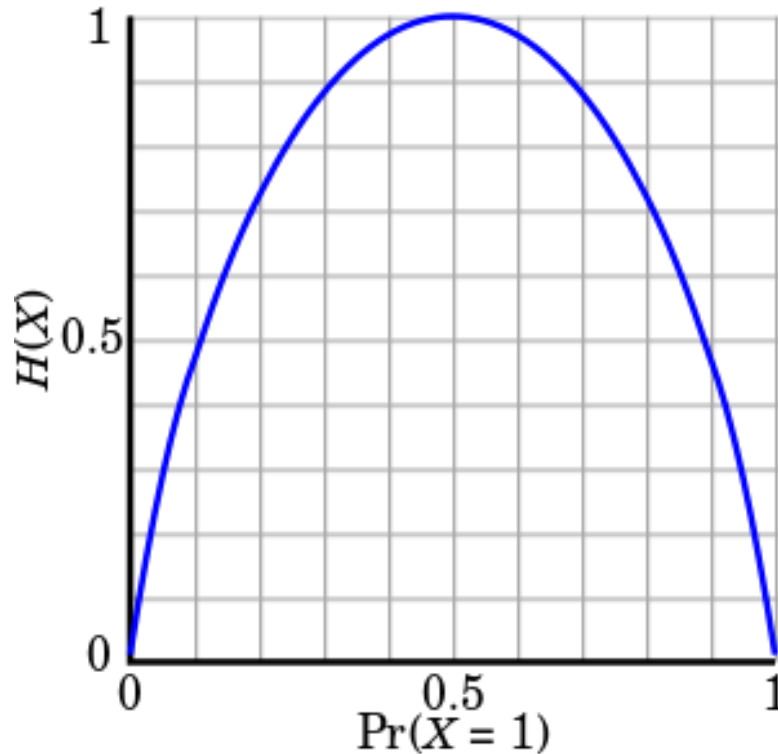
The expected amount of information we get from observing a random variable.

Let a discrete random variable be drawn from distribution p take on one of k possible values with probabilities $p_1 \dots p_k$

$$\begin{aligned} H(p) &= \sum_{i=1}^k p_i I(x_i) \\ &= \sum_{i=1}^k p_i \log_2 \frac{1}{p_i} \\ &= - \sum_{i=1}^k p_i \log_2 p_i \end{aligned}$$

Entropy Example

Plot of entropy vs. coin toss “fairness”



Maximum fairness =
maximum entropy

Completely biased =
minimum entropy

Image source: Wikipedia, by Brona and Alessio Damato

Cross Entropy

Entropy is the minimum number of bits needed to communicate some message, *if we know what probability distribution the message is drawn from.*

Cross entropy is for when we don't know.

e.g., language is drawn from some true distribution, the language model we train is an approximation of it

$$H(p, q) = - \sum_{i=1}^k p_i \log_2 q_i$$

p : “true” distribution

q : model distribution

Estimating Cross Entropy

When evaluating our LM, we assume the test data is a good representative of language drawn from p .

So, we estimate cross entropy to be:

$$H(p, q) = -\frac{1}{N} \log_2 q(w_1 \dots w_N)$$

True language
distribution, which
we don't have
access to.

Language model
under evaluation

Size of test corpus
in number of tokens

The words in the
test corpus

Perplexity

Cross entropy gives us a number in bits, which is sometimes hard to read. Perplexity makes this easier.

$$\text{Perplexity}(p, q) = 2^{H(p, q)}$$

Warm-Up Exercise

Evaluate the given unigram language models using perplexity:

A B C B B

Model 1

$$P(A) = 0.3$$

$$P(B) = 0.4$$

$$P(C) = 0.3$$

Model 2

$$P(A) = 0.4$$

$$P(B) = 0.5$$

$$P(C) = 0.1$$

$$\text{Perplexity}(p, q) = 2^{H(p, q)}$$

$$H(p, q) = -\frac{1}{N} \log_2 q(w_1 \dots w_N)$$

Why MLE?

Why is using relative frequencies of events in the training set a reasonable thing to do?

Why is it called maximum likelihood estimation?

Are there other choices for estimating model parameters?

What is Maximum Likelihood?

Find the model parameters that maximize the likelihood of (i.e., the probability of generating) the training corpus, X .

Find θ^{MLE} s.t. $P(X; \theta^{MLE})$ is maximized

In language modelling:

- Words are random variables that are drawn from a categorical probability distribution
- For a given context, they are **i.i.d.** (independently, and identically distributed)

Categorical Random Variables

1-of-K discrete outcomes, each with some probability

e.g., coin flip, die roll, draw a word from a language model

Probability of a training corpus, $C = x_1, x_2, \dots, x_N$:

$$\begin{aligned} K = 2: P(C; \theta) &= \prod_{n=1}^N P(x_n; \theta) \\ &= \theta^{N_1} (1 - \theta)^{N_0} \end{aligned}$$

Can similarly extend for $K > 2$

Notes:

- When $K=2$, it is called a Bernoulli distribution
- Sometimes incorrectly called a multinomial distribution, which is something else

Maximizing Quantities

Calculus to the rescue!

Take derivative and set to 0.

Trick: maximize the log likelihood instead (math works out better)

- This is okay because log is a monotonically increasing function, so it doesn't affect the location of the maximum.

MLE Derivation for a Bernoulli

Maximize the log likelihood:

$$\begin{aligned}\log P(C; \theta) &= \log(\theta^{N_1}(1 - \theta)^{N_0}) \\ &= N_1 \log \theta + N_0 \log(1 - \theta)\end{aligned}$$

$$\begin{aligned}\frac{d}{d\theta} \log P(C; \theta) &= \frac{N_1}{\theta} - \frac{N_0}{1 - \theta} = 0 \\ \frac{N_1}{\theta} &= \frac{N_0}{1 - \theta}\end{aligned}$$

Solve this to get:

$$\theta = \frac{N_1}{N_0 + N_1}$$

Or,

$$\theta = \frac{N_1}{N}$$

MLE Derivation for a Categorical

The above generalizes to the case where $K > 2$.

Parameters are now $\theta_0, \theta_1, \theta_2, \dots, \theta_{K-1}$

Counts are now $N_0, N_1, N_2, \dots, N_{K-1}$

Note: Need to add a constraint that $\sum_{i=0}^{K-1} \theta_i = 1$ to ensure that the parameters specify a probability distribution.

Use the method of *Lagrange multipliers*

Hint:
$$L(\theta) = \log P(C; \theta) - \lambda \left(\sum_{i=0}^{K-1} \theta_i - 1 \right)$$

$$L(\theta) = \sum_i N_i \log \theta_i - \lambda (\sum_i \theta_i - 1)$$

Find partial derivatives $\partial L / \partial \theta_i$ and $\partial L / \partial \lambda$, set them to zero and solve resulting system of equations

MLE Derivation for a Categorical

1. $L(\theta) = \log P(C; \theta) - \lambda(\sum_{i=0}^{K-1} \theta_i - 1)$ Plug in likelihood of training corpus,
2. $L(\theta) = \sum_i N_i \log \theta_i - \lambda(\sum_i \theta_i - 1)$ apply log laws
3. $\frac{\partial L}{\partial \theta_i} = \frac{N_i}{\theta_i} - \lambda = 0 \rightarrow N_i = \lambda \theta_i$ There are K of these equations.
4. $\frac{\partial L}{\partial \lambda} = -(\sum_i \theta_i - 1) = 0 \rightarrow \sum_i \theta_i = 1$
5. $\sum_i N_i = \sum_i \lambda \theta_i$ Sum up all K equations from Step 3
6. $N = \lambda$ By (4) and (5). Remember sum of all counts is N.
7. $N_i = N \theta_i$ By (3) and (6).
8. $\theta_i = \frac{N_i}{N}$

Steps

1. Gather a large, representative training corpus
2. Learn the parameters from the corpus to build the model
3. **Once the model is fixed, use the model to evaluate on testing data**

Overfitting

MLE often gives us a model that is too good of a fit to the training data. This is called **overfitting**.

- Words that we haven't seen
- The probabilities of the words and N-grams that we have seen are not representative of the true distribution.

But when testing, we evaluate the LM *on unseen data*. Overfitting lowers performance.

Out Of Vocabulary (OOV) Items

Suppose we train a LM on the WSJ corpus, which is about economic news in 1987 – 1989. What probability would be assigned to *Brexit* at test time? What would be the perplexity of the test corpus?

In general, we know that there will be many words in the test data that are not in the training data, no matter how large the training corpus is.

- Neologisms, typos, parts of the text in foreign languages, etc.
- Remember Zipf's law and the long tail

Explicitly Modelling OOV Items

During training:

- Pick some frequency threshold
- All vocabulary items that occur less frequently are replaced by an <UNK>
- Now, treat <UNK> as another vocabulary item

During testing:

- Any unseen words are called <UNK>

Smoothing

Training corpus does not have all the words

- Add a special <UNK> symbol for unknown words

Estimates for infrequent words are unreliable

- Modify our probability distributions

Smoothe the probability distributions to shift probability mass to cases that we haven't seen before or are unsure about

Smoothed Parameter Estimation

Smoothing means we are no longer doing MLE. We now have some **prior belief** about what the parameters should be like.

MLE:

Find θ^{MLE} s.t. $P(X; \theta^{MLE})$ is maximized

We are finding a parameter estimate that maximizes this combination of data likelihood and prior.

Smoothed:

Find θ^{Smthd} s.t. $P(X; \theta^{Smthd})P(\theta^{Smthd})$ is maximized

Add- δ Smoothing

Modify our estimates by adding a certain amount to the frequency of each word. (sometimes called **pseudocounts**)

e.g., unigram model

$$P(w) = \frac{\text{Count}(w) + \delta}{|Lexicon| * \delta + |Corpus|}$$

Pros: simple

Cons: not the best approach; how to pick δ ? Depends on sizes of lexicon and corpus

When $\delta = 1$, this is called **Laplace discounting**

Add- δ Smoothing in General

For a categorical distribution with k possible outcomes and a training corpus of C trials:

$$P(outcome) = \frac{\text{Count}(outcome) + \delta}{C + k * \delta}$$

So, what would be the formula for a bigram model, $P(w_t|w_{t-1})$?

Without smoothing:

$$P(w_t|w_{t-1}) = \frac{\text{Count}(w_{t-1}, w_t)}{\text{Count}(w_{t-1})}$$

How to modify it?

$$P(w_t|w_{t-1}) = \frac{\text{Count}(w_{t-1}, w_t) + \delta}{\text{Count}(w_{t-1}) + |Lexicon| * \delta}$$

Example

Suppose we have a LM with a vocabulary of 20,000 items.

In the training corpus, we see donkey 10 times.

- Of these, in 5 times it was followed by the word kong.
- In the other 5 times, it was followed by another word.

What is the MLE estimate of $P(\textit{kong} | \textit{donkey})$?

What is the Laplace estimate of $P(\textit{kong} | \textit{donkey})$?

Example

Suppose we have a LM with a vocabulary of 20,000 items.

In the training corpus, we see donkey 10 times.

- Of these, in 5 times it was followed by the word kong.
- In the other 5 times, it was followed by another word.

What is the MLE estimate of $P(\textit{kong} | \textit{donkey})$?

5/10

What is the Laplace estimate of $P(\textit{kong} | \textit{donkey})$?

6 / 20010

Interpolation

In an N-gram model, as N increases, data sparsity (i.e., unseen or rarely seen events) becomes a bigger problem.

In an **interpolation**, use a lower N to mitigate the problem.

Simple Interpolation

e.g., combine trigram, bigram, unigram models

$$\begin{aligned}\hat{P}(w_t|w_{t-2}, w_{t-1}) &= \lambda_1 P^{MLE}(w_t|w_{t-2}, w_{t-1}) \\ &\quad + \lambda_2 P^{MLE}(w_t|w_{t-1}) \\ &\quad + \lambda_3 P^{MLE}(w_t)\end{aligned}$$

Need to set $\sum_i \lambda_i = 1$ so that the overall sum is a probability distribution

How to select λ_i ? Compare different values on your development set!

Good-Turing Smoothing

A more sophisticated method of modelling unseen events (usually N-grams)

Remember Zipf's lessons

- We shouldn't adjust all words/N-grams uniformly.
- The frequency of a word or N-gram is related to its rank—we should be able to model this!
- Unseen N-grams should behave a lot like N-grams that only occur once in a corpus
- N-grams that occur a lot should behave like other N-grams that occur a lot.

Count of Counts

Let's build a histogram to count how many events occur a certain number of times in the corpus.

Event frequency	# events with that frequency
1	$f_1 = 3993$
2	$f_2 = 1292$
3	$f_3 = 664$
...	...

- For some event in bin f_c , that event occurred c times in the corpus; c is the numerator in the MLE.
- **Idea:** re-estimate c using f_{c+1}

Good-Turing Smoothing Defined

Let N be total number of observed event-tokens, w_c be an event that occurs c times in the training corpus.

$$N = \sum_i f_i \times i$$

$$P(UNK) = f_1 / N \quad \leftarrow \text{Note that } P(UNK) \text{ is for all unseen events}$$

$$\text{Then: } c^* = \frac{(c+1)f_{c+1}}{f_c}$$

$$P(w_c) = c^* / N \quad \leftarrow \text{Note that } P(w_c) \text{ is for one event that occurs } c \text{ times}$$

Example:

Let N be 100,000.

Word frequency	# word-types
1	$f_1 = 3,993$
2	$f_2 = 1,292$
3	$f_3 = 664$
...	...

$$\begin{aligned} P(UNK) &= 3993 / 100000 \\ &= 0.03993 \\ &\text{(for all unseen events)} \\ &\text{(Divide by total estimated} \\ &\text{number of unseen word} \\ &\text{types.)} \end{aligned}$$

$$\begin{aligned} c_1^* &= 2 * 1292 / 3993 \\ &= 0.647 \end{aligned}$$

$$\begin{aligned} c_2^* &= 3 * 664 / 1292 \\ &= 1.542 \end{aligned}$$

Exercises

Suppose we have the following counts:

Word	ship	pass	camp	frock	soccer	mother	tops
Freq	5	3	2	2	1	1	1

Give the MLE and Good-Turing estimates for the probabilities of:

- any unknown word
- *soccer*
- *camp*
- *pass*

$$N = \sum_i f_i \times i$$
$$c^* = \frac{(c+1)f_{c+1}}{f_c}$$
$$P(UNK) = f_1 / N$$
$$P(w_c) = c^* / N$$

Good Turing smoothing for Bigram

$$C^*(w_t, w_{t-1}) = [C(w_t, w_{t-1}) + 1] * \frac{f_{C(w_t, w_{t-1})+1}}{f_{C(w_t, w_{t-1})}}$$

$$\hat{P}(w_t | w_{t-1}) = \lambda_1 P^{MLE}(w_t | w_{t-1}) + \lambda_2 P^{MLE}(w_t)$$

Good-Turing Refinement

As seen above, simple Good-Turing fails for higher values of c , because f_{c+1} is often 0.

Solution: Estimate f_c as a function of c

- We'll assume that a linear relationship exists between $\log c$ and $\log f_c$
- Use linear regression to learn this relationship:
$$\log f_c^{LR} = a \log c + b$$
- For lower values of c , we continue to use f_c ; for higher values of c , we use our new estimate f_c^{LR} .

Model Complexity Trade-Offs

In general, there is a trade-off between:

- model expressivity; i.e., what trends you could capture about your data with your model
- amount of training data needed to give a good estimate of the parameters of the model

If you use an expressive model (e.g., high values of N in N -gram modelling), it is easier to overfit, and you need to do smoothing and use more data.

OTOH, if your model is too weak, your performance will suffer as well.